

2. Add regulatAPIParams (views.schema.json)

```
2182 +
2183
2184     "regulatAPIParams": {
2185       "type": "object",
2186       "description": ">: Regulat API Action Params",
2187       "required": [
2188         "actionType"
2189       ],
2190       "anyOf": [
2191         {
2192           "properties": {
2193             "actionType": {
2194               "enum": [
2195                 "get_aml_scan"
2196               ]
2197             }
2198           },
2199           "required": [
2200             "entityParams"
2201           ]
2202         },
2203         {
2204           "properties": {
2205             "actionType": {
2206               "type": "string",
2207               "description": "M: action type when main actionType is regulat_api",
2208               "enum": [
2209                 "get_aml_scan"
2210               ]
2211             },
2212             "entityParams": { ...
2213           }
2214         }
2215       ]
2216     }
2217   }
2218 }
```

3. Define entityParams under regulatAPIParams (views.schema.json)

```
2210 "entityParams": {
2211   "type": "object",
2212   "description": "?: entity parameters",
2213   "required": [
2214     "codice_azienza",
2215     "id_anagrafica",
2216     "nome",
2217     "cognome",
2218     "yob",
2219     "id_somministrazione",
2220     "tipo_soggetto"
2221   ],
2222   "additionalProperties": false,
2223   "properties": {
2224     "codice_azienza": {
2225       "description": "M: Codice azienda",
2226       "type": "string"
2227     },
2228     "id_anagrafica": {
2229       "description": "M: Id anagrafica object of verification",
2230       "type": "number"
2231     },
2232     "nome": {
2233       "description": "M: Name or company name object of verification",
2234       "type": "string"
2235     },
2236     "cognome": {
2237       "description": "M: Surname object of verification",
2238       "type": "string"
2239     },
2240     "yob": {
2241       "description": "M: Year of birth object of verification",
2242       "type": "number"
2243     },
2244     "id_somministrazione": {
2245       "description": "M: The verify to be filled",
2246       "type": "number"
2247     },
2248     "tipo_soggetto": {
2249       "description": "M: Entity type object of verification",
2250       "type": "string"
2251     }
2252   }
2253 }
```

4. Create interface regulat_api_params.ts

```
src > app > oc > interfaces > TS regulat_api_params.ts > ...
You, 24 seconds ago | 1 author (You)
1+ export interface RegulatAPIParams {
2   |   actionType: 'get_aml_scan',
3   |   entityParams?: {
4   |     |   codice_azienda: string,
5   |     |   id_anagrafica: number,
6   |     |   nome: string,
7   |     |   cognome: string,
8   |     |   yob: number,
9   |     |   id_somministrazione: number,
10  |     |   tipo_soggetto: string
11  |   }
12  | };
You, 3 days ago via PR #453 *
```

5. Add export path

```
src > app > oc > interfaces > TS index.ts
18 export * from './user_info.interface';
19 export * from './illogger_service.interface';
20 export * from './display_service_constants.interface';
21 export * from './display_service_settings.interface';
22 export * from './marker_replacer.interface';
23 export * from './import_item.interface';
24 export * from './import_list.interface';
25 export * from './export_item.interface';
26 export * from './export_list.interface';
27 export * from './report_item.interface';
28 export * from './report_list.interface';
29 export * from './scroll_info.interface';
30 export * from './time_tracker_status.interface';
31 export * from './field.interface';
32 export * from './email_action_parameters';
33 export * from './email_body_key_parameters';
34 export * from './search_toggle';
35 export * from './google_api_params';
36 export * from './regulat_api_params';
37 export * from './menu_option.interface';
38 export * from './selection_action.interface';
39 export * from './selection_action_params.interface';
40 export * from './restrictions.interface';
41 export * from './widgets_configuration.interface';
42 export * from './file_manager_view_type.interface';
43
```

6. Add in fieldInputEvent in field.interface.ts

```
src > app > oc > interfaces > TS field.interface.ts > * FieldInputEvent
You, 6 minutes ago | 7 authors (jzorjnz and others)
1 | import { GoogleAPIParams, RegulatAPIParams, MenuOption } from ".";
2 | import { FormWidgetType } from "../types";
3 | import { EmailActionParameters } from "../email_action_parameters";
4 |
5 | [jzorjnz, 20 months ago | 2 authors (jzorjnz and others)]
6 | export interface Validator {
7 |   message: string;
8 |   name: string;
9 |   validator: string;
10 | value?: string;
11 | }
12 |
13 | [jzorjnz, 24 months ago | 1 author (jzorjnz)]
14 | export interface Item {
15 |   id: string;
16 |   name: string;
17 | }
18 |
19 | [jzorjnz, 24 months ago | 2 authors (jzorjnz and others)]
20 | export interface Key {
21 |   name: string;
22 |   inputType: string;
23 | }
24 |
25 | You, 6 minutes ago | 3 authors (jzorjnz and others)
26 | export interface FieldInputEvent { [jzorjnz, 23 months ago * Added Field Input Event. ...]
27 |   actionType: 'hide' | 'show' | 'readOnly' | 'query' | 'query_style' | 'show_message' | 'toggle' | 'update' | 'update_style' | 'update_time_tracker' | 'google_api' | 'regulat_api';
28 |   eventName: string;
29 |   values: any[];
30 |   condition: 'equalTo' | 'notEqualTo' | 'greaterThan' | 'lessThan' | 'none';
31 |   queryFunc?: string;
32 |   styleAttribute?: 'background_color' | 'font_color';
33 |   updateFunc?: string;
34 |   actionTarget?: {
35 |     name: string;
36 |     type: 'table' | 'form' | 'tableform';
37 |     keymap?: [
38 |       {
39 |         source: string;
40 |         destination: string
41 |       }
42 |     ]
43 |   };
44 |   message?: {
45 |     messageText: string;
46 |     actionOnYes: {
47 |       actionType: 'reload' | 'query' | 'email',
48 |       queryFunc?: string,
49 |       emailActionParameters?: EmailActionParameters
50 |     },
51 |     actionOnNo: {
52 |       actionType: 'reload' | 'query',
53 |       queryFunc?: string
54 |     }
55 |   };
56 |   googleAPIParams?: GoogleAPIParams;
57 |   RegulatAPIParams?: RegulatAPIParams
58 | }
```

NB: After the 6 points I get this:

```
Failed to compile.
src/app/oc/interfaces/field.interface.ts:1:27 - error TS2305: Module '"."' has no exported member 'RegulatAPIParams'.
1 import { GoogleAPIParams, RegulatAPIParams, MenuOption } from ".";
```

ERROR: Module “.” has no exported member ‘RegulatAPIParams’.

/* So for now I commented it, to make the test work. The file where I commented some code are:

```
TS field.interface.ts src/app/oc/inter... M
TS index.ts src/app/oc/interfaces M
```

The commit has been named “Commented import interface”.

*/

Second Step: manage the flow in the backend

1. Add event in form-getter.component.ts (line 1381) which calls runRegulatEvent.

```
src > app > oc > views > form-getter > TS form-getter.component.ts > FormGetterComponent > eventCallback
817
818
819 // callback for pubSub events, value has form of {origin, index, valueSet, data}
820 private eventCallback(event: any, value: any, keyListener: string): void {
821     const _this = this;
822     // console.table(event);
823     // console.table(value);
824     // console.table(_this.filteredFormData);
825     // console.log(`keyListener: ${keyListener}`);
826     _this._console.log(event, value, keyListener);
827
828     // check if this is a formRowProperties event
829 > if (event.actionType === 'showRow' && event.condition === 'equalTo') { ...
885 > }
886 // not a ViewProperties event, check the condition if any -- TODO: support other conditions beyond equalTo
887 let conditionMet = true;
888
889 > if (event.condition !== null) { ...
905 > }
906
907 > if (event.actionType === 'show' || event.actionType === 'hide' || event.actionType === 'toggle') { ...
934 > } else if (event.actionType === 'readOnly') { ...
953 > } else if (event.actionType === 'reload' && conditionMet) { ...
958 > } else if (event.actionType === 'update_time_tracker') { ...
960 > } else if (event.actionType === 'navigate' && conditionMet) { ...
1006 > } else if ((event.actionType === 'query' || event.actionType === 'query_style' || event.actionType === 'combo_lazy_loading') && conditionMet) { ...
1159 > } else if ((event.actionType === 'update' || event.actionType === 'update_style') && conditionMet) { ...
1183 > } else if (event.actionType === 'show_message' && conditionMet && event.message) { ...
1379 > } else if (event.actionType === 'google_api') { ...
1381 > } else if (event.actionType === 'regulat_api') {
1382 |     _this.runRegulatEvent(event, value, keyListener);
1383 | }
      |
      | Syed Zeeshan Akhtar, 3 months ago via PR #430 • Implemented Create Drive API
```

2. Create async function (first draft) related to the button in form-getter

```
1641 ~ async runRegulatEvent(event, value, keyListener) { // You, just month + first step: regulatAPIParams
1642 ~   let _this = this;
1643 ~   const regulatAPIParams: RegulatAPIParams = event.RegulatAPIParams;
1644 ~   let formValues = _this.formArray.first.form.value;
1645 ~
1646 ~   // process the booleans (1/0 instead of true/false)
1647 ~   for (const value in formValues) {
1648 ~     // ...
1649 ~   }
1650 ~
1651 ~   if(!regulatAPIParams || !regulatAPIParams.actionType) {
1652 ~     _this._toastService.showErrorToast("Missing Regulat API params");
1653 ~   }
1654 ~   else {
1655 ~     const codiceAziendaAML = formValues[regulatAPIParams.entityParams.codice_aziende];
1656 ~     const idAnagraficaAML = formValues[regulatAPIParams.entityParams.id_anagrafica];
1657 ~     const nomeAML = formValues[regulatAPIParams.entityParams.nome];
1658 ~     const cognomeAML = formValues[regulatAPIParams.entityParams.cognome];
1659 ~     const yobAML = formValues[regulatAPIParams.entityParams.yob];
1660 ~     const idSomministrazioneAML = formValues[regulatAPIParams.entityParams.id_somministrazione];
1661 ~     const tipoSoggettoAML = formValues[regulatAPIParams.entityParams.tipo_soggetto];
1662 ~     if(regulatAPIParams.actionType == "get_aml_scan") {
1663 ~       if(!regulatAPIParams.entityParams) {
1664 ~         _this._toastService.showErrorToast("Missing Regulat API entity params");
1665 ~       }
1666 ~       else {
1667 ~         _this._dialogService.showLoadingDialog('Scanning in progress', 'Please wait...');
1668 ~
1669 ~         let connected_registries = await _this.backendService.getConnectedRegistries(codiceAziendaAML, idAnagraficaAML).toPromise();
1670 ~         console.log(connected_registries);
1671 ~
1672 ~         let connectedRegistries = connected_registries.response; //array which contains the result of a query in 2 different columns:
1673 ~
1674 ~         /* for(let i = 0; i < anagraficaFolders.length; i++) {
1675 ~           let scan_contents = await _this.backendService.getAmlScan(codiceAziendaAML, idAnagraficaAML, connected_registries[i], nomeAML, cognomeAML, yobAML, idSomministrazioneAML, tipoSoggettoAML).toPromise();
1676 ~           console.log(scan_contents);
1677 ~         } */
1678 ~
1679 ~         let scan_contents = await _this.backendService.getAmlScan(codiceAziendaAML, idAnagraficaAML, connectedRegistries, nomeAML, cognomeAML, yobAML, idSomministrazioneAML, tipoSoggettoAML).toPromise();
1680 ~         console.log(scan_contents);
1681 ~
1682 ~         _this._dialogService.closeDialog();
1683 ~         _this._toastService.showSuccessToast('Successfully Scanned!'); // show success toast
1684 ~       }
1685 ~     }
1686 ~     else {
1687 ~       _this._toastService.showErrorToast("Missing Google API Get Email Thread Params");
1688 ~     }
1689 ~   }
1690 ~ }
```

Is it the best way to do it? I have to loop here or in the lambda?

3. Created the methods in backend.service.ts

```
428 ~ getConnectedRegistries(company: string, registry: string): Observable<any> {
429 ~   this.amplifyService.auth();
430 ~   this.myGetInit.queryStringParameters = { request_type: 'getConnectedRegistries', company: company, registry: registry };
431 ~   return from(this.amplifyService.api().get(this.apiName, '/' + this.regulatVPCApiName, this.myGetInit));
432 ~ }
433 ~
434 ~ getAmlScan(company: string, registry: number, connected_registries: number [], name: string, surname: string, yob: number, checkId: number, entytyType: string): Observable<any> {
435 ~   this.amplifyService.auth();
436 ~   this.myGetInit.queryStringParameters = { company: company, registry: registry, connected_registries: connected_registries, name: name, surname: surname, yobi: yob, checkId: checkId, entytyType: entytyType };
437 ~   return from(this.amplifyService.api().get(this.apiName, '/' + this.regulatApiName, this.myGetInit));
438 ~ }
```

Third Step: Create lambda functs (regulat and regulat_VPC)

1. Index.js of regulat

Here I do the operation to get the scan result in json format, and when the process is complete the lambda regulat_VPC will be called. Flow: Get acces token (bearer), define which api have to be called (depends on entityType) and get scan result. Then, if the connectedRegistries isn't void, loop all of element of the array and do another scan. The scan results have to be store in scannedData.

<https://github.com/alacritascompliance/onecompliance/blob/c82990932b1fc9534f6079dc96f2db0631540b91/lambda/regulat>

NB: I need to pass also the const "company", "registry" and "checkId" of the "original" registry with the scannedData in the payload of lambda.invoke(regulat_VPC). I need to add also the requestType='getAmlScan' in the payload.

2. Index.js of regulat_VPC

From here basically I run this query, passing the calling payload by regulat.lambda

```
// run process query
query = `select entrasp.process_aml_scans('${scans}')`;
response = await client.query(query);
```


Fourth Step: Database

1. Table aml_scans on imports schema

```
CREATE TABLE IF NOT EXISTS imports.aml_scans
(
    codice_azienda character varying(30) COLLATE pg_catalog."default",
    id_scan character varying(50) COLLATE pg_catalog."default" NOT NULL,
    id_anagrafica numeric(12,0) NOT NULL,
    date_of_scan date,
    entity_type character(1) COLLATE pg_catalog."default",
    id_somministrazione numeric(12,0),
    scan_data json,
    CONSTRAINT aml_scans_pkey PRIMARY KEY (id_scan)
)
WITH (
    OIDS = FALSE
)
TABLESPACE pg_default;

ALTER TABLE IF EXISTS imports.aml_scans
    OWNER to postgres;

GRANT ALL ON TABLE imports.aml_scans TO postgres;

GRANT ALL ON TABLE imports.aml_scans TO rdsadmin;
```

Where in the “scan” attribute I put the response “data” in json format.

2. entrasp.process_aml_scans(scans json) function

Insert into entrasp.aml_scans the scan result. Then loop for each new scan and insert into risposte.